

تقرير المرحلة الرابعة – محرك الحجز (الطلبات والإيجارات) (Phase 4) (Booking Engine: Requests & Rentals)

اسم المرحلة ورقمها

المرحلة الرابعة: محرك الحجز (Requests & Rentals) – Phase 4، وهي أعلى مراحل المشروع خطورةً حتى الآن (Highest-Risk Phase) لاحتوائها على ضمانة القفل الذري (Atomic Locking) لمنع حجز نفس السرير مرتين في نفس الوقت.

الهدف من المرحلة

بناء تدفق كامل من الطلب إلى الإيجار (Request → Rental) بضمانة صارمة: لا يمكن أبدًا لطالبي الحصول على نفس السرير في نفس الوقت. الرحلة: الطالب يطلب سريرًا → السرير يُقفل مؤقتًا (pending) → صاحب المبنى يراجع الطلب ويتواصل مع الطالب هاتفياً خارج النظام → صاحب المبنى يوافق أو يرفض من لوحة التحكم → الموافقة تُنشئ سجل إيجار (Rental) ويصبح السرير occupied؛ الرفض أو انتهاء المهلة يعيد السرير إلى available.

الملفات والمجلدات التي اتعملت فعليًا

1. مودول الطلبات (Requests Module) – جديد بالكامل

src/modules/requests/

— request.model.js

→ student, bed, building/owner_id (deno

└ request.repository.js	→ عدة البيانات، بما فيها استعلام الدفعات
└ request.service.js	→ لمكررة، الموافقة/الرفض/انتهاء الصلاحية
└ request.controller.js	→ الطالب وصاحب المبنى endpoints
└ request.routes.js	
└ request-expiry.job.js	→ scheduler.core مسجلة (في مهمة مجدولة)

2. موديول الإيجارات (Rentals Module) — تم بناء المحتوى (الهيكل كان placeholder فارغ)

src/modules/rentals/	
└ rental.model.js	→ student, bed, building/owner_id (denormalized)
└ rental.repository.js	
└ rental.service.js	→ إنشاء الإيجار عند الموافقة، تدفق الإخلاء
└ rental.controller.js	
└ rental.routes.js	

3. ملفات معدلة في موديولات موجودة من مراحل سابقة

src/config/constants.config.js	→ REQUEST_STATUS و RENTAL_STATUS إضافة
src/modules/audit/audit.model.js	→ (null) بقى اختياريًا actor حقل
src/modules/audit/audit.service.js	→ act بقت تقبل writeAuditLog()
src/modules/beds/bed.repository.js	→ conditionalUpdateStatus إضافة
src/modules/beds/bed.service.js	→ (في) atomicTransition() إضافة
src/modules/beds/bed.controller.js	→ ط قبل حذف أي سرير (البند 11)
src/modules/apartments/apartment.controller.js	→ حذف أي شقة (البند 11)
src/modules/buildings/building.controller.js	→ ف أي مبنى (البند 11)
src/modules/kyc/kyc.repository.js	→ (يعني) findByStudentIds() إضافة
src/modules/kyc/kyc.service.js	→ (يعني) getKycMapForStudents() إضافة
src/modules/students/student.repository.js	→ (دفعي) findByIds() إضافة
src/modules/students/student.service.js	→ (دفعي) getStudentRecordBy إضافة
src/modules/students/student.controller.js	→ (دفعي) getFullProfileForO إضافة
src/modules/students/student.routes.js	→ GET /api/students/ إضافة

src/app.entry.js

→ /api/requests و /api/rent

src/server.entry.js

→ انتهاء صلاحية الطلبات في الـ

.github/workflows/backend-tests.yml

→ من 20 إلى 22 Node.js ترقية

4. ملفات الاختبار

tests/integration/booking-engine.test.js

→ بل في قسم الاختبارات)

tests/integration/students-kyc.test.js

→ describe b تحديث الـ

شرح تفصيلي لكل خطوة اتفدت

الخطوة 1: نموذج الطلب (request.model.js)

يحتوي على: `bed / student` (مراجع)، `owner_id / building` (منسوخة من السريير وقت الإنشاء — نفس نمط الـ `denormalization` من المرحلة الثالثة)، `status` (يستخدم `REQUEST_STATUS` الموجود بالفعل من المرحلة الأولى)، `rejection_reason` (enum من أربع قيم محددة كما طلب النص)، `move_in_date`، `note` حر، و `expires_at` (محسوبة ومخزنة وقت الإنشاء = وقت الإنشاء + 48 ساعة، بدلاً من إعادة حسابها في كل تشغيل لمهمة انتهاء الصلاحية). فهرس مركب على `{status, expires_at}` يدعم استعلام الدُفعات لمهمة انتهاء الصلاحية مباشرة.

الخطوة 2: منطق القفل الذري (الجزء الأهم في المرحلة كلها)

تم بناء `bed.repository conditionalUpdateStatus(bedId, expectedStatus, newStatus)` استدعاء واحد لـ `findOneAndUpdate` بشرط مطابقة `({_id: bedId, status: expectedStatus})`. هذه العملية ذرية على مستوى المستند في MongoDB نفسه — لو حاول طالبان طلب نفس السريير في نفس اللحظة، MongoDB يضمن أن استدعاء واحد فقط من الاثنين يطابق الشرط وينفذ التعديل؛ الآخر يرجع `null` فوراً لأن حالة السريير تكون قد تغيرت بالفعل. لا توجد قفلة على مستوى التطبيق (Application Lock) ولا

طابور رسائل (Message Queue) في هذه العملية على الإطلاق — قاعدة
البيانات نفسها هي القفلة، تمامًا كما يطلب النص.

فوق هذه الدالة مباشرة، بُني `bed.service.atomicTransition()` كغلاف يضيف تسجيل التغيير في `bed-history.service` عند النجاح فقط. هذه الدالة تُستخدم في كل انتقال لحالة السيرير في هذه المرحلة: إنشاء الطلب (`available→pending`)، الموافقة (`pending→occupied`)، الرفض/انتهاء الصلاحية (`pending→available`)، وإنهاء الإيجار (`occupied→available`).

الخطوة 3: مهمة انتهاء صلاحية الطلبات (`request-expiry.job.js`)

مُسجَّلة في `scheduler.core` (المبني في المرحلة صفر) بجدولة كل 15 دقيقة، وتُستدعى فعليًا عند بدء تشغيل السيرفر عبر `server.entry.js`. تعالج الطلبات على دفعات (100 طلب في كل استعلام)، وتعيد الاستعلام من الصفر بعد كل دفعة (بدل استخدام `limit/skip` على نتيجة متحركة، لأن إنهاء صلاحية طلب يزيله من مجموعة "طلبات معلقة ومنتبهة الصلاحية" نفسها، فاستخدام `skip` هنا كان سيتخطى أو يكرر صفوفًا بين الدفعات). كل طلب داخل الدفعة يُعالج في `try/catch` منفصل، فشل طلب واحد لا يوقف باقي الدفعة.

الخطوة 4: نقطة الطلبات المعلقة لصاحب المبنى (مع ملخص بيانات الطالب)

`GET /api/requests/pending` بترجّع الطلبات المعلقة الخاصة بصاحب المبنى (مُصفاة على مستوى الاستعلام بـ `owner_id`)، ومع كل طلب ملخص بروفایل و **KYC الطالب** مرفق. لتفادي مشكلة N+1 (البند 4.4 من CLAUDE.md)، تم بناء دالتين دُفعيتين جديدتين: `kycService.getKycMapForStudents()` و `studentService.getFullProfilesWithKycForIds()` — استعلامان اثنان فقط (طلاب + KYC، كلاهما بـ `$in`) بغض النظر عن عدد الطلبات في الصفحة، وليس استعلامًا لكل طلب.

الخطوة 5: الموافقة والرفض (`Confirm/Reject`)

الموافقة (`request.service.confirmRequest`): تحقق من أن الطلب `pending`، تحويل السيرير ذريًا `pending→occupied`، إنشاء سجل إيجار عبر `rental.service.createRentalFromRequest`، تحديث حالة الطلب إلى

REQUEST_STATUS.APPROVED (انظر قسم القرارات التقنية لسبب استخدام APPROVED بدل قيمة "confirmed" جديدة)، وتسجيل الحدث في سجل التدقيق.

الرفض (request.service.rejectRequest): تحقق من أن السبب المُرسَل من ضمن القيم الأربع المحددة، تحويل السرير ذريًا pending→available ، تحديث حالة الطلب إلى REJECTED مع حفظ السبب والملاحظة.

الخطوة 6: حد الطلبات المكررة لكل طالب

request.service.createRequest يتحقق أولاً من عدد طلبات الطالب المعلقة حاليًا (countPendingForStudent)، ويرفض الطلب (409) لو كان العدد ≤ 2. هذا فحص عادي (count-then-create) وليس جزءًا من الضمانة الذرية — تم توضيح هذا الفرق صراحةً في تعليقات الكود: أسوأ سيناريو هنا هو طالب بثلاث طلبات معلقة بدل اثنين (تجاوز بسيط لقاعدة عمل ناعمة)، وهذا مختلف جوهريًا عن حجز نفس السرير لطلابيين مختلفين (وهو ما تحميه الضمانة الذرية الحقيقية في الخطوة 2).

الخطوة 7: تدفق الإخلاء (Vacating/Move-Out) على مستوى الإيجار وليس السرير

rental.service.markVacating() : يحوّل حالة الإيجار إلى vacating من غير أي لمسة للسرير — يبقى occupied طوال هذه الفترة، تمامًا كما صحّح المستند المرفق. rental.service.finalizeMoveOut() : يحوّل السرير ذريًا occupied→available ويقفل الإيجار (closed)، مسموح استدعاؤها مباشرة من حالة active (بدون المرور بـ vacating أولاً) — قرار تقني موضّح أدناه.

الخطوة 8: الـ Waitlist (اختياري)

لم يُبنَ — الخطوة المذكورة صراحةً كـ "(Optional)" في نص المرحلة، ولم يُطلب تنفيذها. لا يُعتبر هذا انحرافًا لأنه اختياري بنص المستند نفسه.

الخطوة 9: تسجيل كل انتقال حالة في سجل التدقيق الحقيقي

كل تحويل حالة (سرير، طلب، إيجار) يُسجّل عبر auditService.writeAuditLog() — إما مباشرة (request_created) ، request_expired ، request_rejected ، request_confirmed

bed- rental_created , rental_vacating , rental_closed) أو عبر service.history (bed_status_change , الذي بدوره غلاف حول نفس writeAuditLog()). لا توجد آلية تسجيل منفصلة في أي مكان.

الخطوة 10: نقطة اطلاع صاحب المبنى على بروفایل الطالب الكامل + الـ KYC (مؤجلة من المرحلة الثانية)

GET /api/students/:studentId/full-profile — مقيّدة لصاحب المبنى فقط، والوصول ليس مطابقة owner_id بسيطة (الطالب لا يملك حق الوصول owner_id أصلاً) بل فحص علاقة حقيقي: هل يوجد طلب معلّق (pending) أو إيجار نشط/في طور الإخلاء (active/vacating) يربط هذا الطالب بهذا صاحب؟ تم تنفيذ الفحصين عبر دالتين مخصصتين
requestService.hasPendingRequestWithOwner ,
rentalService.hasActiveRelationshipWithOwner والجمع بينهما بـ OR، مع رفض صريح (403) لو فشل الاثنان معاً. تم كتابة الاختبار الحرج المطلوب صراحةً بالنص: صاحب مبنى A لا يستطيع رؤية بيانات طالب لا تربطه به أي علاقة (انظر قسم الاختبارات).

الخطوة 11: تحديث قيد حذف Buildings/Apartments/Beds من المرحلة الثالثة

المرحلة الثالثة استخدمت 'available' != bed.status كمؤشر مؤقت لـ "وجود علاقة نشطة" لعدم وجود موديول Rentals وقتها. الآن بعد وجوده، تم تحديث منطق الحذف في **الثلاثة مستويات** (سرير/شقة/مبنى) ليتحقق أولاً من وجود إيجار نشط/في طور الإخلاء (rentalService.bedHasActiveRental / anyBedHasActiveRental)، وهذا الفحص الآن هو الإشارة الرسمية (Authoritative). فحص حالة السرير القديم لم يُحذف — بقي كطبقة حماية ثانوية (مثلاً: سرير في وضع maintenance بدون أي إيجار لا يزال محميًا من الحذف بنفس منطق المرحلة الثالثة).

قرار تصميمي مهم هنا: هذا الفحص الجديد وُضع في طبقة الـ controller (bed/apartment/building)، وليس داخل bed.service.js نفسها — التفاصيل والسبب (تجنّب اعتمادية دائرية بين الموديولات) موضحة في قسم القرارات التقنية أدناه.

الخطوة 12: اختبار التزامن الحرج

انظر قسم الاختبارات — هذا هو التسليم الأهم في المرحلة بأكملها حسب تعليمات المستخدم الصريحة.

أي قرارات تقنية اتأخذت أثناء التنفيذ

1. استخدام `REQUEST_STATUS.APPROVED` الموجود بالفعل بدل قيمة `"confirmed"` جديدة: نفس منطق قرار المرحلة الثالثة بخصوص `BED_STATUS` — تجنّب قاموسين متعارضين لنفس المفهوم. `REQUEST_STATUS` كان معرّفًا من المرحلة الأولى ويحتوي بالفعل على `APPROVED` بنفس المعنى المقصود بـ `"confirmed"` في نص هذه المرحلة.
2. جعل حقل `actor` في `audit.model.js` اختياريًا (`nullable`): مهمة انتهاء صلاحية الطلبات فعل آلي بلا فاعل بشري. بدل اختراع مستخدم "نظام" وهمي لإرضاء قيد "مطلوب دائمًا"، تم السماح بـ `actor: null` مع اسم فعل واضح (`request_expired`) يوضّح بذاته أن هذا فعل تلقائي. سجل التدقيق يبقى كاملاً وواضحًا (نعرف ماذا حدث ومتى بدقة) — فقط بدون فاعل بشري مزيف.
3. فحص الإيجار النشط عند الحذف يعيش في طبقة الـ `controller`، وليس داخل `bed.service.js`: كان الخيار الأول أن تستدعي `bed.service.deleteBed()` مباشرة `rentalService.bedHasActiveRental()` لكن `rental.service.js` نفسه يستدعي `bed.service.js` (لعملية `atomicTransition` عند إنهاء الإيجار) — لو استدعى `bed.service` رجوعًا إلى `rental.service`، كانت النتيجة اعتمادية دائرية (**Circular Dependency**) بين الموديولين، وهو ما يخالف بنية "الموديول المستقل" (البند 7.2 من CLAUDE.md) ويُنتج سلوكًا غير موثوق في Node.js (تصدير جزئي حسب ترتيب التحميل). الحل: نقل الفحص إلى الـ `controllers` (`bed/apartment/building`)، اللي أصلاً وظيفتها تنسيق أكثر من موديول — كل `controller` يستدعي `rentalService` و `bedService`

buildingService / apartmentService بدون أي مشكلة، لأن لا أحد يستدعي ال controllers رجوعًا.

4. **السماح بإنهاء الإيجار (finalizeMoveOut) مباشرة من حالة active ، من غير المرور بـ vacating أولاً:** نص المرحلة يصف تدفقًا من خطوتين (vacating ثم إنهاء)، لكنه لا يمنع صراحةً إنهاء الإيجار مباشرة لو غادر الطالب فجأة من غير إشعار مسبق. vacating تبقى إشعارًا اختياريًا مسبقًا، مش خطوة إلزامية في المسار.

5. **جدولة مهمة انتهاء الصلاحية كل 15 دقيقة:** النص يحدد نافذة الانتهاء نفسها (48 ساعة) لكن لا يحدد دورية تشغيل المهمة. تم اختيار 15 دقيقة كتوازن بين سرعة الاستجابة (طلب منتهي الصلاحية لا يبقى معلقًا لفترة طويلة) وعدم إثقال قاعدة البيانات باستعلامات متكررة جدًا.

6. **بناء studentService.getFullProfileWithKyc() و getFullProfilesWithKycForIds() من الصفر:** نص المرحلة يفترض أن studentService.getFullProfileWithKyc(studentId) "موجودة بالفعل من المرحلة الثانية" — لكنها لم تكن موجودة فعليًا (المرحلة الثانية أُجّلت ال endpoint بالكامل فلم تكن هناك حاجة لبنائها وقتها). تم بناؤها الآن، وتوثيق هذا صراحةً في تعليقات الكود كتصحيح لافتراض المستند، وليس اكتشاف خطأ حقيقي في الكود القديم.

7. **ترقية Node.js في CI من 20 إلى 22:** طلب صريح من صاحب المشروع لحل تحذير الإهمال (Deprecation Warning) المتكرر في كل تشغيلات GitHub Actions منذ المرحلة صفر.

الاختبارات التي اتعملت والنتائج

تم إنشاء tests/integration/booking-engine.test.js (بالإضافة لتحديث students-kyc.test.js) ويغطي:

- **اختبار التزامن الحرج (الأهم في المرحلة كلها):** إطلاق 10 طلبات لنفس السيرير في نفس اللحظة تقريبًا (Promise.all) من 10 طلاب مختلفين،

والتحقق من أن **واحدًا بالضبط** نجح (201) والباقي رُفض (409) — مع تأكيد مزدوج من قاعدة البيانات نفسها: عدد سجلات الطلب لهذا السرير = 1، وحالة السرير = pending (وليس أي حالة أخرى فاسدة). اختبار إضافي للترامن المتسلسل (طلب ثانٍ لسرير معلق بالفعل يُرفض)، واختبار لرفض طلب سرير غير متاح أصلاً (occupied).

- **حد الطلبات المكررة:** طالب يقدر يفتح طلبين معلقين بس، والثالث يُرفض (409) — مع تأكيد أن السرير الثالث لم يُقفل.
- **تدفق الموافقة:** تحويل السرير لـ occupied، إنشاء إيجار active، تحديث حالة الطلب لـ approved؛ رفض الموافقة على طلب غير معلق (409)؛ رفض موافقة صاحب مبنى B على طلب صاحب مبنى A (403).
- **تدفق الرفض:** سبب رفض صحيح يعيد السرير لـ available مع حفظ السبب؛ رفض سبب غير صالح (422)؛ إمكانية طلب نفس السرير مرة أخرى بعد الرفض.
- **مهمة انتهاء الصلاحية:** طلب منتهي الصلاحية فعليًا يتحول لـ expired ويُحرَّر السرير مع سجل تدقيق بفاعل null؛ **اختبار حماية من التسابق (race):** طلب تمت الموافقة عليه بالفعل لا يُلمَس حتى لو expires_at يتأخر في الماضي؛ اختبار معالجة الدُفعات (5 طلبات منتهية الصلاحية دفعة واحدة).
- **تدفق الإخلاء والإيجار:** vacating لا تغيّر حالة السرير (يبقى occupied)؛ إنهاء الإيجار يحرر السرير ويقفل الإيجار؛ إمكانية الإنهاء المباشر من active بدون المرور بـ vacating؛ رفض تعليم إيجار مُقفل بالفعل كـ vacating (409)؛ إمكانية طلب السرير من جديد بعد إنهاء الإيجار.
- **قائمة الطلبات المعلقة لصاحب المبنى:** التأكد من وجود ملخص بروفایل/KYC الطالب مرفق مع كل طلب، واحترام الـ Pagination.
- **الاختبار الحرج الصريح المطلوب بنص المرحلة (الخطوة 10):** صاحب مبنى A لا يستطيع رؤية بيانات KYC لطالب لا تربطه به أي علاقة (403) — بالإضافة لاختبارات إيجابية (وصول ناجح عبر طلب معلق، وعبر إيجار نشط)، واختبار لعلاقة منتهية/مرفوضة فقط (لا طلب معلق ولا إيجار نشط) ترجع 403 أيضًا.
- **اختبارات قيد الحذف المُحدَّث (البند 11):** حذف سرير/شقة/مبنى فيه إيجار نشط مرفوض (409) برسالة خطأ تذكر "rental" صراحةً (تأكيد أن المسار الجديد هو اللي اشتغل، مش القديم فقط)؛ إمكانية الحذف بعد إنهاء الإيجار؛

التأكد أن الفحص القديم (حالة maintenance بلا إيجار) لسه شغال كطبقة حماية ثانوية.

- **اختبارات حدود الأدوار (Role Guards):** رفض توكن owner على إنشاء طلب (403)، رفض توكن student على قائمة الطلبات المعلقة والإيجارات (403)، رفض طالب يحاول يشوف بروفایل طالب ثاني (403 لأن ال endpoint owner-only)، ورفض الوصول من غير توكن على كل ال endpoints الجديدة (401).

نتيجة التشغيل الفعلي: [سيتم تحديث هذا القسم فور رفع الكود والتحقق من نتيجة GitHub Actions الحقيقية — راجع قسم "الحالة النهائية للمرحلة" أدناه]. تم التحقق محليًا من: خلو كل الملفات الجديدة والمعدلة من أخطاء syntax (node - - check)، ونجاح تحميل شجرة الموديولات بالكامل فعليًا (require('./src/'))، وبدون أي خطأ اعتمادية دائرية، والتأكد من أن كل الدوال الحرجة (app.entry.js) ، createRequest ، createRentalFromRequest ، atomicTransition) ، runExpirySweep ، getFullProfilesWithKycForIds (مُصدّرة وقابلة للاستدعاء فعليًا. التشغيل الفعلي للاختبارات نفسها لم يتم محليًا في بيئة ال sandbox، لنفس القيد الموثّق سابقًا (تعدّر تحميل ثنائي MongoDB اللازم لـ -mongodb memory-server بسبب قيود شبكة ال GitHub Actions — sandbox) يبقى الطريقة الوحيدة الموثوقة للتحقق، كما في كل مرحلة سابقة.

أي انحراف عن الخطة الأصلية

لا يوجد انحراف جوهري عن نص المرحلة المُصحّح

النسخة المرفقة من phase-4-booking-engine.md كانت مُصحّحة بالفعل من صاحب المشروع قبل التنفيذ (تصحيح قيم BED_STATUS ، وتوضيح أن vacating تعيش على الإيجار لا السرير) — الخطوات 1-12 كلها اتفقت كما هي، عدا الخطوة 8 (Waitlist) المذكورة صراحة كاختيارية.

انحرافات ثانوية (تقنية، موثّقة في قسم القرارات أعلاه)

- إعادة استخدام REQUEST_STATUS.APPROVED بدل قيمة "confirmed" جديدة (قرار 1).

- جعل `audit.actor` اختياريًا لدعم الأفعال الآلية (قرار 2).
- نقل فحص الإيجار عند الحذف لطبقة الـ `controller` بدل الـ `service`، لتفادي اعتمادية دائرية حقيقية بين موديولي `beds` و `rentals` (قرار 3) — هذا القرار الوحيد اللي ممكن يُعتبر "انحراف بنيوي" عن الافتراض الضمني إن كل منطق يعيش في الـ `service`، لكنه ضروري تقنيًا وموثق بالكامل في الكود والتقارير.
- السماح بإنهاء الإيجار مباشرة من `active` (قرار 4) — تفصيل غير محدد صراحةً بالنص.
- بناء `studentService.getFullProfileWithKyc()` من الصفر بدل افتراض وجودها (قرار 6) — تصحيح لافتراض في نص المرحلة نفسه، وليس خطأً في كود سابق.

الحالة النهائية للمرحلة

الحالة: الكود جاهز بالكامل (الموديولات، الـ routes، الاختبارات، وربط المهمة المجدولة)، ولسه غير مُتحقق منه فعليًا عبر GitHub Actions — لا يجوز اعتبارها "منتهية" حتى تظهر نتيجة CI حقيقية.

كل خطوات التنفيذ (1-7، 9-12) اتنفذت، الخطوة 8 (Waitlist) اختيارية ولم تُن. اجتاز كل ملف فحص الـ `syntax`، وتأكدنا من تحميل شجرة الموديولات كاملة بدون أخطاء اعتمادية دائرية. لسه معلق: **الدفع (push) والتحقق عبر GitHub Actions**، بسبب مشكلة `T:\sakanify\.git\index.lock` المتكررة والموثقة من المراحل السابقة (نفس السلوك: الـ `sandbox` يقدر يُنشئ القفل لكن مش يقدر يحذفه، والحل معروف — إغلاق أي برنامج Git على جهاز المستخدم وحذف الملف يدويًا).

سيتم تحديث هذا التقرير (قسم "الاختبارات اللي اتعملت والنتائج" و"الحالة النهائية للمرحلة") فور حل مشكلة القفل ورفع نتيجة CI الحقيقية.